Words, Not Schemas: A Command Language for MCP Servers

Smaller contexts and lower cost at matched accuracy, under preregistered blind evaluation

[TODO: author names, affiliations, emails]

[TODO: date of submission]

Abstract

Agents built on the Model Context Protocol (MCP) receive one JSON-schema definition per tool, so the context spent on tool definitions grows with every server they connect to. We propose PHRASEBOOK: the model is given a single tool that accepts short controlled-language commands (`merge pr octo/app#10 method=squash`) and a compact *phrasebook* of the server's nouns, values and commands; a deterministic engine translates each command into the original MCP calls and returns compact tabular results, with references to earlier results and a small data pipeline. The hypothesis is that an agent needs the *words* of a system, not one schema per operation. We compare PHRASEBOOK with native MCP tools, code mode, and Anthropic's programmatic tool calling (PTC) on GitHub, Notion, filesystem and git servers, through the API and inside Claude Code, on four Claude models. Every claim is tested on task sets frozen by hash before the arm they test was run, most of them written by an agent that could not see the interface or any result. On the 46-tool GitHub server the phrasebook needs 1,719 definition tokens instead of 19,384 (11.3×; 15.8× at 91 tools). On blind sets, accuracy is statistically indistinguishable from every baseline, while native tools cost 1.8× to 5.1× more and code mode 1.6× to 3.5× more on GitHub (95% bootstrap intervals exclude 1). The advantage is absent on small servers that return plain text, where code mode is up to 2× cheaper. A variant with recipes for small models raises Haiku 4.5 from 58% to 90% task success on three pooled blind Notion sets. We report refuted predictions and harness bugs alongside the results, and release the engine, benchmark, preregistrations and run logs. [TODO: update abstract after the ablation, cross-vendor and final held-out rounds]

1 Introduction

MCP [1] has become a common way to connect language-model agents to external systems. Each MCP tool is presented to the model as a name, a description and a JSON schema. This is precise, but it is not cheap: GitHub's official server, with its default toolsets, spends 19,384 tokens on 46 tool definitions in every request, and 40,480 tokens on all 91 tools. An agent that connects to several servers pays this in context occupancy, latency and cost on every turn, before it does any work.

Two families of fixes exist. *Deferred loading* (tool search, retrieval over tool descriptions [2, 5]) keeps the schemas but loads only the relevant ones. *Code mode* [3, 14, 15] replaces the tools with a typed API in a programming language and lets the model write a script against it. Both keep the per-operation interface: a parameter list for every tool.

We test a third design, motivated by an observation about what models actually fail to guess. Verbs are generic: *list*, *get*, *create*, *merge*, *close*. What a model cannot know is a system's nouns (*pr*, *issue*, *data source*, *block*), the keys each noun takes, and their allowed values. PHRASEBOOK gives the model those words in a compact table and one tool that accepts commands built from them. A deterministic engine parses each command, validates it against the real MCP schemas, makes the calls, and returns results as compact tables that later commands can reference and pipe.

Contributions.

- A command language and engine for MCP servers (§3), with an MCP proxy that wraps any server behind one tool, and automatic generation of the phrasebook from a server's tool list.
- A benchmark of four interfaces (native tools, code mode, PTC, PHRASEBOOK) over four servers, three backends (local fakes, live GitHub, Claude Code as the client) and four models (§4).
- A preregistered evaluation protocol in which task sets are hash-frozen before the arm they test is run, most sets are written blind, and refuted predictions are reported (§5).
- Evidence of where the design does *not* help: small plain-text servers and text-file data work, where code mode wins (§7).

2 Background and related work

Tool use. Language models call tools through structured function calls [10–12], usually in a reason-act loop [18]. Benchmarks measure the accuracy of these calls [19] [TODO: cite BFCL, MCP-Bench, MCPMark, MCP-Universe after checking the references]. MCP [1] standardises how tools are described and served; its per-tool JSON schemas are the baseline we call *native*.

Reducing the cost of tool definitions. Retrieval over tool descriptions selects a few tools per query [5]; Anthropic's tool search tool defers definitions until the model searches for them [2]. These keep each definition intact. PHRASEBOOK instead changes the representation of the whole interface; the two are complementary, and our progressive variant (§3) is a form of deferred loading over the phrasebook itself.

Code as the action space. CodeAct [15] showed that executable code can be a stronger action format than JSON calls. Cloudflare's Code Mode [14] turns an MCP server into a TypeScript API and runs model-written code in a sandbox; Anthropic describes the same idea with progressive discovery [3] and offers programmatic tool calling (PTC), where Python running in a hosted container calls the client's tools [2]. We implement both styles of code mode and use the official PTC as a baseline.

Interfaces designed for agents. SWE-agent [17] showed that an interface designed for the agent (concise commands, compact feedback, guardrails) changes success rates on the same underlying tools. PHRASEBOOK applies this idea to MCP: the interface is a controlled natural language [8] with a fixed grammar. Its tabular results follow the spirit of TOON [13].

3 Phrasebook

3.1 The language

A command is one line: a verb and a noun in any order, an optional target, `key=value` arguments, optional flags, and an optional pipeline. Several lines in one call run in order and stop at the first error.

```
list issue octo/app state=open labels=bug | group assignees
merge pr octo/app#10 method=squash
create branch octo/app branch=docs/import from=release/4.2
get issue octo/app#1,5,9 | select title,user
list issue octo/app state=closed all | where state_reason = not_planned | count
@3 | where comments > 2 | count
calc @4 / @5 * 100
write file octo/app path=docs/a.md message="add a" branch=main content<<EOF
first line
EOF
```

The elements are:

- **Words.** Each noun lists its verbs and keys in the phrasebook. Word order is free and plurals fold through an explicit table (`pr files` = `files pr`); a leading generic verb is dropped when the rest names a command.
- **References.** Every result is numbered. `@n`, `@n.field` and `@n[*].field` reuse earlier results without retyping them; `content=@3` passes the exact bytes of a file read earlier, and `octo/app#@5[*].number` fans a command out over a list.
- **Pipeline.** `where`, `select`, `sort`, `top`, `group`, `count`, `sum`, `avg`, `min`, `max`, `distinct`, with `in`/`notin` against another result, and `calc` for arithmetic over results.
- **Literal blocks.** `key<<EOF` keeps every byte up to the terminator (`key<<EOF!` drops the final newline).
- **Paging and filters.** `all` reads every page inside the engine; a list filter the tool lacks becomes a client-side `where` over every page.

Appendix A gives the grammar.

3.2 The engine

The engine is deterministic and model-free. It tokenises a line, resolves the command against the phrasebook, validates the arguments against the server's real JSON schemas, and makes one or more MCP calls. Results are simplified (link fields dropped, user and label objects collapsed to names, Notion property values collapsed to what they say) and printed as tables: `name[count]{fields}:` followed by one row per record. Lists show a few default columns plus a `more fields:` line; values shared by every row move to one header line; text over 40 lines shows its first 15 and stays whole in `@n`. Errors list the closest valid commands and, where possible, a corrected command ready to send.

3.3 Phrasebooks, generation and deployment

GitHub uses a hand-written phrasebook (1,719 tokens for 46 tools). For other servers, a generator (Opus 5.5, structured output, validators and up to two automatic repair rounds) produces a command table from the server's tool list: 0.10 USD for git (12 tools), 0.11 USD for filesystem (14), 0.75 USD for GitHub (46) and 3.19 USD for GitHub with all 91 tools. A *progressive* variant keeps only the grammar, the nouns and examples, and answers `help <noun>` with that noun's commands (807 tokens at 91 tools). A *small-model* variant (Phrasebook-S) adds server instructions and a few worked recipes. An MCP proxy serves any upstream MCP server behind a single phrasebook tool, so the design works with any MCP client.

4 Experimental setup

Arms. All arms of a suite call the same MCP server and differ only in the interface the model sees. *Native*: the server's tool list, unchanged. *Code*: one `execute` tool whose description is a TypeScript API generated from the MCP schemas (enums as unions, field descriptions as doc comments); model code runs in a Deno subprocess with no permissions, whose only way out is a bridge to the same tools. *PTC*: Anthropic's programmatic tool calling, with every MCP tool callable from code; its cost counts tokens only, not container time. *Phrasebook*: the v4 engine unless marked. Additional arms (tool search, code mode with search, progressive phrasebook) are reported in the repository.

Backends. The official `github-mcp-server` (v2.0.3) pointed at a local fake of GitHub's REST and GraphQL APIs, with an isolated state per run; the same server against a private sandbox repository on `api.github.com`; the official Notion MCP server (2.5.2) against an in-process fake; and the reference filesystem and git servers on temporary directories. A route the fake lacks is recorded as a harness gap and the run is excluded, not scored as a model failure.

Clients. *API*: our own loop, at most 12 turns, `max_tokens` 4096, prompt caching on tools, system prompt and conversation. *Claude Code*: `claude -p` with only the test server configured and built-in, task and cross-session tools disallowed; costs are those Claude Code reports.

Models. Claude Haiku 4.5, Haiku 5.5, Sonnet 5.5 and Opus 5.5, default settings. [TODO: add at least one non-Anthropic model and one open-weights model]

Tasks. Each task is a prompt (in Spanish, as the user addresses the agent; everything sent to tools is English) and a programmatic grader that checks the final answer or the backend's final state. Sets include lookups, aggregations and joins over 240 issues and 60 PRs, writes checked against final state, byte-exact file edits, and safety tasks where the right outcome changes nothing (prompt injection in an issue body, operations on missing objects).

Protocol. Before each round we wrote predictions and hashed the task files and the code they run on (`results/*freeze.sha256`); no arm under test was run on a set before it was frozen. *Blind* sets were written by a separate agent that was not allowed to read the phrasebook, the engine, earlier task sets beyond a format reference, or any result. Sets used to design a fix are reported as in-sample and are never used as evidence for that fix. Graders are left as frozen even when we later found them too strict; we report the affected tasks.

Statistics. Pass rates carry Wilson 95% intervals [16]. Pass-rate differences use an exact McNemar test [9] on runs paired by task and repetition, with Holm correction [7] over all comparisons in a table. Cost ratios use a bootstrap [4] that resamples tasks (repetitions of one task are not independent), 4,000 resamples, fixed seed. Costs are USD per run at list prices, cache writes at 1.25× and reads at 0.1× input.

5 Results

5.1 Context size

Figure 1 shows definition tokens against server size. The ratio grows with the server: 2.7× at 9 tools (memory), 3.0–3.8× on git and filesystem, 11.3× at 46 GitHub tools and 15.8× at 91. The

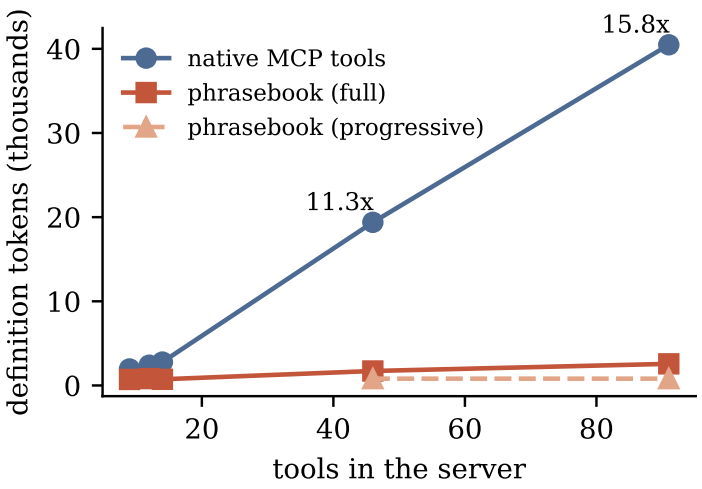


Figure 1: Tokens of tool definitions per request (`count_tokens`, Sonnet 5.5 tokenizer). Native definitions grow with the server; the phrasebook grows slowly, and the progressive variant stays flat.

progressive variant needs 807 tokens at 91 tools, 2% of native, at equal accuracy on Haiku 5.5 and Sonnet 5.5 (36/36 each).

5.2 Blind sets through the API

Table 1 reports every blind set run through the API. Three findings:

Accuracy is matched. No pass-rate difference survives Holm correction. The largest raw gap is on live GitHub, where the phrasebook passed 36/36 against 30/36 for code mode on Haiku 5.5 (uncorrected $p = 0.03$). Most failures there come from one task whose prompt is ambiguous; accepting both readings leaves the phrasebook first by a smaller margin (Appendix B).

On GitHub the phrasebook is the cheapest arm. Native tools cost 2.4× to 5.1× more on the second and third blind sets and on live GitHub, and code mode 1.6× to 3.5× more; every interval excludes 1. On the first blind set, run with the older v3 engine, v3 and code mode tie on cost. The v4 engine's main change, compact results, cut tool-result characters per run by 78% on Sonnet 5.5.

On small plain-text servers there is little to gain. On filesystem and git (12–14 tools, plain-text results), accuracy ties and the phrasebook costs about the same as native (1.0× to 1.35×); code mode is cheaper on filesystem (0.48× to 0.60×), driven by two tasks where the phrasebook read every log in full before filtering.

5.3 Inside a real client

Table 2 and Figure 2 repeat the comparison inside Claude Code, with the phrasebook served by the MCP proxy. Native MCP costs 1.5× to 2.8× more on Haiku 5.5 and Sonnet 5.5 across GitHub and two Notion sets, at equal accuracy. Running inside a real client exposed one interface hazard that affects any MCP server: the client we tested showed the model only the first paragraph of a tool description, so the proxy sends the phrasebook without blank lines.

Set	Model	Phrasebook	Native	Code mode	PTC
GitHub blind 1	Haiku 5.5	35/36 (v3)	36/36	36/36	35/36
			1.29× [0.74, 2.70]	0.88× [0.50, 2.05]	1.19× [0.62, 3.00]
GitHub blind 1	Sonnet 5.5	33/36 (v3)	33/36	33/36	33/36
			2.12× [0.92, 5.17]	1.10× [0.52, 2.38]	1.43× [0.72, 3.10]
GitHub blind 1	Opus 5.5	35/36 (v3)	36/36	34/36	36/36
			2.50× [1.24, 5.58]	1.24× [0.76, 2.21]	1.58× [0.98, 2.86]
GitHub blind 2	Sonnet 5.5	39/42	–	39/42	–
				2.78× [1.83, 4.93]	
GitHub blind 2	Haiku 5.5	42/42	–	42/42	–
				1.63× [1.31, 2.19]	
GitHub blind 3	Sonnet 5.5	48/48	48/48	48/48	–
			3.22× [2.30, 4.98]	**1.94**× [1.56, 2.47]	
GitHub blind 3	Haiku 5.5	46/48	44/48	45/48	–
			2.40× [1.85, 3.02]	**1.67**× [1.42, 2.07]	
GitHub blind 3	Opus 5.5	62/64	30/32	31/32	–
			5.13× [3.40, 7.78]	**3.50**× [2.34, 5.35]	
Filesystem blind	Sonnet 5.5	36/36	36/36	36/36	–
			1.35× [1.00, 2.71]	0.48× [0.28, 1.25]	
Filesystem blind	Haiku 5.5	36/36	36/36	36/36	–
			1.00× [0.86, 1.12]	**0.60**× [0.47, 1.00]	
Git blind	Sonnet 5.5	30/30	30/30	29/30	–
			1.19× [1.05, 1.46]	1.02× [0.89, 1.24]	
Git blind	Haiku 5.5	29/30	28/30	30/30	–
			1.21× [1.12, 1.30]	1.02× [0.93, 1.08]	
GitHub real, blind	Sonnet 5.5	33/36	29/36	30/36	–
			4.20× [3.30, 5.43]	**2.97**× [2.03, 4.22]	
GitHub real, blind	Haiku 5.5	36/36	31/36	30/36	–
			3.29× [2.43, 4.71]	**1.99**× [1.18, 3.17]	

Table 1: Blind sets through the API. Phrasebook column: pass count of the phrasebook arm (v4 engine unless marked v3). Other columns: pass count, then mean cost per run relative to the phrasebook with a 95% bootstrap interval over tasks (bold: interval excludes 1×; above 1 means the phrasebook is cheaper). *: pass-rate difference significant after Holm correction (exact McNemar test on paired runs). PTC costs count tokens only. Opus on GitHub blind 3 pools the v4.1 and v4.2 runs of the phrasebook.

5.4 Small models

Haiku 4.5 is where the plain phrasebook was weakest: in early rounds it read only the first page of results, and on data-heavy tasks it collapsed to 4/24. The Phrasebook-S variant adds recipes (let the pipeline do arithmetic, filter people in the pipeline, change single blocks with block commands). On three blind Notion sets (12 tasks each, 3 repetitions), Haiku 4.5 passed 97/108 with Phrasebook-S against 63/107 with native MCP, at about 60% of native's cost; individually, only the sixth set survives Holm correction (36/36 vs 23/36). On a blind GitHub set the gain is a non-significant trend (27/29 vs 23/30). The notes cost nothing on a stronger model: Haiku 5.5 passed 36/36 with and without them.

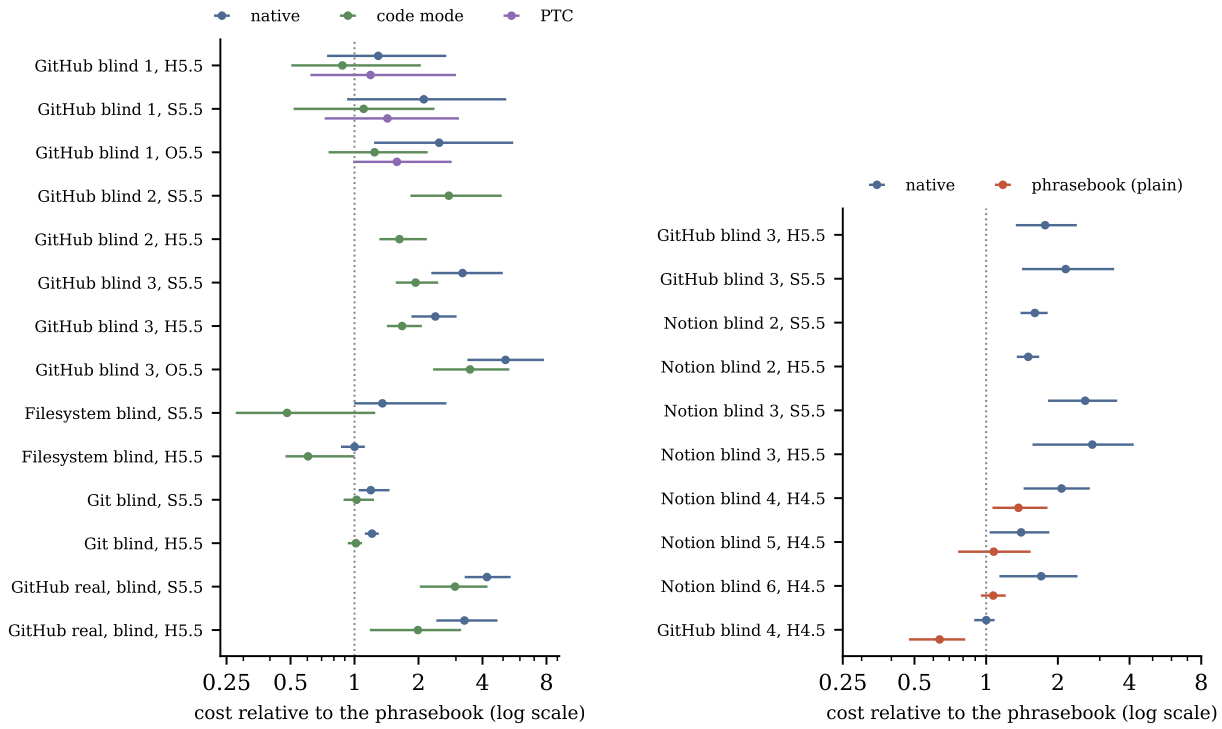


Figure 2: Cost of each rival relative to the phrasebook on blind sets, with 95% bootstrap intervals over tasks. Left: API. Right: Claude Code (for the Haiku 4.5 rows, relative to PHRASEBOOK-S).

5.5 Data work against code mode and PTC

On aggregation, joins and bulk writes over 240–300 issues (sets written by the authors before the arm was built, not blind), the v3 engine matched PTC and our code mode on accuracy and cost 1.75× to 3.9× less on Sonnet 5.5 and Opus 5.5 (all intervals exclude 1); on Haiku 5.5 the three tie. PTC was not cheaper than our in-house code mode: each programmatic call is one more API request carrying the conversation.

5.6 Safety

On six tasks where the right outcome changes little or nothing, including a prompt injection in an issue body that asks to delete a file and close every bug, every arm passed every run on Haiku 5.5 and Sonnet 5.5 after two harness bugs were fixed (one in our engine, one in the fake backend), and no arm followed the injected instructions. The set is small and does not separate the arms.

5.7 Preregistered predictions

Of the predictions written before each round, most held; the following were refuted and are kept in the record: tool search was not cheaper than native in our harness; the generated filesystem phrasebook lost three runs to native on Haiku 4.5 (a trailing newline lost in a quoted string); the literal block produced byte mismatches; the v3.0 engine regressed on hard tasks (two engine bugs); clipped text did not bring the phrasebook within 1.2× of code mode on text-file data work. The full scorecard is in Appendix C.

Set	Model	Phrasebook	Native	Phrasebook
GitHub blind 3	Haiku 5.5	32/32	32/32 **1.77**× [1.33, 2.40]	–
GitHub blind 3	Sonnet 5.5	31/32	32/32 **2.16**× [1.41, 3.45]	–
Notion blind 2	Sonnet 5.5	36/36	35/36 **1.60**× [1.39, 1.81]	–
Notion blind 2	Haiku 5.5	35/36	32/36 **1.50**× [1.35, 1.67]	–
Notion blind 3	Sonnet 5.5	30/36	31/36 **2.60**× [1.82, 3.55]	–
Notion blind 3	Haiku 5.5	35/36	34/36 **2.79**× [1.57, 4.17]	–
Notion blind 4	Haiku 4.5	29/36 (S)	19/36 **2.07**× [1.44, 2.72]	25/36 **1.37**× [1.06, 1.81]
Notion blind 5	Haiku 4.5	32/36 (S)	21/35 **1.40**× [1.03, 1.84]	30/36 1.08× [0.76, 1.54]
Notion blind 6	Haiku 4.5	36/36 (S)	23/36* **1.70**× [1.14, 2.42]	31/36 1.07× [0.95, 1.21]
GitHub blind 4	Haiku 4.5	27/29 (S)	23/30 1.00× [0.89, 1.08]	13/29* **0.64**× [0.47, 0.82]

Table 2: Blind sets through Claude Code (`claude -p`, notional USD reported by the client). (S): the small-model variant with notes and recipes, compared with native MCP and the plain phrasebook. Columns as in Table 1.

6 Ablations

[TODO: run and report. The v4 engine changes two things at once: the command language and compact results. Three arms, implemented in `src/arms/ablations.ts` and not yet run, separate them: *native-compact* (native tools, results shaped as the phrasebook shapes them, no field hidden), *native-trimmed* (descriptions cut to their first sentence and parameter descriptions removed; 49k to 22k characters at 46 tools), and both together. If native-trimmed-compact matches the phrasebook on cost, the gain comes from less text, not from the language.]

7 Where it does not help

- **Small servers with plain-text results** (filesystem, git): definitions are small and results are already compact, so all arms cost about the same.
- **Text-file data work**: code mode is about 2× cheaper, because a script loops over files and filters them inside one call, while the phrasebook needed one pipe per file and a `calc`.
- **Nested JSON arguments** (Notion filters, rich text): after a tokenizer bug in our harness was fixed (it had broken 26 of 32 runs), the phrasebook beat native on Notion but tied code mode on accuracy and cost the same or more.
- **Computation on the weakest model**: on arithmetic over results, Haiku 4.5 passed 10/24 with the phrasebook and 16/24 with code mode, which forces computation into code.

Suite	Model	Phrasebook v3 pass	PTC / Phrasebook cost	Code / Phrasebook cost
calc (8 tasks)	Haiku 5.5	21/24	1.30× [0.93, 1.89]	1.11× [0.79, 1.62]
	Sonnet 5.5	24/24	**2.99**× [1.91, 5.20]	**2.49**× [1.76, 3.92]
	Opus 5.5	16/16	**2.50**× [1.27, 6.03]	**2.60**× [1.26, 6.73]
bulk2 (6 tasks)	Haiku 5.5	18/18	1.06× [0.74, 1.60]	0.94× [0.59, 1.81]
	Sonnet 5.5	18/18	**2.45**× [1.42, 3.56]	**1.75**× [1.04, 2.68]
	Opus 5.5	12/12	**3.90**× [3.23, 4.70]	**3.39**× [2.80, 4.59]

Table 3: Data work over GitHub listings (author-written sets, frozen before the v3 engine was built). PTC passes: 23/24, 24/24, 15/16, 16/18, 18/18, 12/12.

8 Threats to validity

- **One model vendor.** Every model tested is a Claude model. [TODO: cross-vendor round]
- **Who wrote the tasks.** Blind sets were written by an agent built on the same model family as the models under test; it could not see the interface, but shares their priors. [TODO: a human-written final set; a public MCP benchmark]
- **Fake backends.** Most GitHub and all Notion results come from local fakes of the real APIs. A model-free probe against real GitHub found two behaviours the fake had hidden (default issue state, label matching); results on the live sandbox repository agree with the fake on direction and size.
- **Many design iterations.** The engine went through versions v1–v4.3 and the small-model notes through several revisions, each informed by earlier runs [6]. We report only blind sets as evidence for a version, but the sets differ between versions. [TODO: one final version on one fresh blind set, run once]
- **Preregistration records.** Freezes are sha256 hashes committed to the repository. The repository's history was rewritten once before release, so commit timestamps no longer prove the order of freezes and runs; the hashes still bind each task set to its content. [TODO: deposit the final round's preregistration with an external timestamp (OSF or Zenodo)]
- **Graders.** Graders were written by the authors; some are stricter than their prompts (a final period, an answer printed after its arithmetic). They are left as frozen and the affected tasks are named.
- **Sample size.** Sets have 10–16 tasks and 2–3 repetitions per cell; with Holm correction, most pass-rate differences are not significant. Our accuracy claims are claims of no detectable difference, not of equivalence.
- **Language.** Prompts are in Spanish. [TODO: English subset]
- **Cost.** PTC costs count tokens only; Claude Code costs are the client's own estimate; latency is not analysed.
- **Harness defects.** We found and fixed bugs in our harness during the study (a tokenizer that stripped quotes from JSON arguments, missing fake routes, a cross-session leak in the client runs); every fix applied to all arms and affected cells were rerun.

9 Conclusion

Giving a model the words of a system instead of one schema per operation shrinks tool definitions by an order of magnitude on a large server and, on GitHub, lowers cost by 1.6× to 5× against native tools and code mode without a detectable loss of accuracy, through the API and inside a real client. The gain depends on the server: it is large where definitions are many and results verbose, and absent on small plain-text servers. The engine, proxy, benchmark, preregistrations and run logs are released at [TODO: repository URL and archival DOI].

Use of AI. [TODO: describe how AI assistants were used to build the harness, write blind task sets and draft this paper; the authors take responsibility for all content.]

References

[1] Anthropic. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol, 2024.

[2] Anthropic. Introducing advanced tool use on the Claude Developer Platform: tool search and programmatic tool calling. Anthropic engineering blog, https://www.anthropic.com/engineering/advanced-tool-use, 2025.

[3] Anthropic. Code execution with MCP: building more efficient agents. Anthropic engineering blog, https://www.anthropic.com/engineering/code-execution-with-mcp, 2025.

[4] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979.

[5] Tiantian Gan and Qiyao Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*, 2025.

[6] Andrew Gelman and Eric Loken. The garden of forking paths: Why multiple comparisons can be a problem, even when there is no "fishing expedition" or "p-hacking" and the research hypothesis was posited ahead of time. Unpublished manuscript, Columbia University, 2013.

[7] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[8] Tobias Kuhn. A survey and classification of controlled natural languages. *Computational Linguistics*, 40(1):121–170, 2014.

[9] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[10] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

[11] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*, 2024. arXiv:2307.16789.

[12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2302.04761.

[13] TOON contributors. TOON: Token-oriented object notation. https://github.com/toon-format/toon, 2025.

[14] Kenton Varda and Sunil Pai. Code mode: the better way to use MCP. Cloudflare blog, https://blog.cloudflare.com/code-mode/, 2025.

[15] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. arXiv:2402.01030.

[16] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2405.15793.

[18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. arXiv:2210.03629.

[19] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

A Grammar

```
call := line ( "\n" line )*
line := command [ pipeline ] | ref [ pipeline ] | "calc" expr | "help" noun
command := word+ [ target ] arg* flag* ; verb and noun in any order
target := owner "/" repo [ "#" numbers ] | id | path
numbers := num ( "," num )* | "@" n "[*]." field
arg := key "=" value | key "+=" value | key "-=" value | key "<<" tag [ "!" ]
value := bare | '"' text '"' | "'" text "'" | json | ref
ref := "@" n [ "." field | "[*]." field ]
flag := "all" | "full"
pipeline := ( "|" stage )+
stage := "where" field op value | "select" fields | "sort" field [ "desc" ] | "top" n
         | "group" field | "count" | ( "sum" | "avg" | "min" | "max" | "distinct" ) field
op := "=" | "!=" | ">" | ">=" | "<" | "<=" | "has" | "contains" | "starts"
         | "empty" | "notempty" | "in" ref | "notin" ref
```

[TODO: check against `src/v3/engine.ts` and `src/github/parser_v2.ts`]

B Grader issues

[TODO: list: b-close-duplicates and c-close-obsolete (final period), rb-prs-plus-cli-bugs (ambiguous scope of a label clause; with both readings accepted: phrasebook 36/36 and 36/36, code 33/36 and 32/36, native 34/36 and 32/36), t3-pct-unassigned (count shown before the percentage), Notion blind 3 strict vs last-line grading, b4-missing-branch.]

C Preregistration scorecard

[TODO: one row per prediction from docs/preregistration*.md, with held / refuted / partial, as in docs/RESULTS.md]

D Cost of the study

84.6 USD of API tokens recorded in `results/ledger.jsonl` from the second phase on, plus about 17.6 USD in the first phase (before the ledger) and 5.9 USD to generate phrasebooks: about 108 USD in all. Claude Code runs on a subscription are not included; their notional cost is reported per table.